

第 39 讲

AdamW 与 Weight Decay

为什么 Transformer / LLM 训练通常写 AdamW，而不是普通 Adam

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 38 讲已经学会 Adam：它用一阶矩 m_t 记录近期 Gradient 的方向，用二阶矩 v_t 记录 Gradient 平方的尺度，再做坐标级自适应更新。本讲进一步解决另一个经常出现在 LLM 训练配置中的词：**weight decay**。真正的难点不是记住 `torch.optim.AdamW`，而是分清三件很容易混在一起的事：**L2 regularization**、普通 SGD 中的 **weight decay**、AdamW 中的 **decoupled weight decay**。

学习目标

学完本讲，你应该能够：

1. 解释为什么训练时有时希望限制参数长期变得过大；
2. 看懂带 L2 penalty 的目标函数 $L_{data} + \frac{\lambda}{2} \|\theta\|_2^2$ ；
3. 从公式推导 L2 penalty 为什么给 Gradient 增加 $\lambda\theta$ ；
4. 解释为什么在**最基础的 SGD** 里，L2 regularization 与 multiplicative weight decay 可以等价；
5. 解释为什么到了 Adam，这种等价关系会被自适应缩放破坏；
6. 逐步解释 AdamW 的 decoupled weight decay 更新；
7. 看懂 PyTorch `Adam(weight_decay=...)` 与 `AdamW(weight_decay=...)` 的语义差异；
8. 理解 Learning Rate schedule 会改变累计 decay 强度；
9. 知道哪些参数常常会被排除在 weight decay 之外，以及为什么这只是 recipe 而不是数学定律；
10. 在 LLM / Agent Systems 论文中正确记录 optimizer、LR、weight decay、parameter groups 等实验条件。

1 先回答最基础的问题：为什么要让 Weight 变小一点？

到目前为止，我们主要考虑的是：怎样让 data loss 下降。假设模型参数是 θ ，普通训练目标是：

$$\min_{\theta} L_{data}(\theta)$$

只要 Training Loss 能更低，优化器就可能愿意把某些 Weight 推得非常大。

通俗解释

可以把参数理解成模型内部可以自由调节的“旋钮”。如果我们只告诉模型“把训练集误差降下来”，它只关心最后是否能把训练样本拟合好，并不会天然在意旋钮是不是被拧得很极端。Regularization 的思想之一，就是在“拟合数据”之外，再给参数行为加一个偏好或约束。

一种最经典的做法是 L2 regularization:

$$J(\theta) = L_{data}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

这里:

- L_{data} : 模型对数据本身的 Loss;
- θ : 所有希望被正则化的参数;
- $\|\theta\|_2^2 = \sum_i \theta_i^2$: 参数平方和;
- λ : regularization strength, 控制“压小参数”的力度;
- 前面的 $\frac{1}{2}$: 只是为了求导后公式更整洁。

2 L2 Regularization 的梯度到底发生了什么?

先从一维参数 w 开始:

$$J(w) = L_{data}(w) + \frac{\lambda}{2} w^2$$

对 w 求导:

$$\frac{dJ}{dw} = \frac{dL_{data}}{dw} + \lambda w$$

也就是说, 原本的 data gradient 是:

$$g_{data} = \frac{dL_{data}}{dw}$$

加上 L2 penalty 后:

$$g_{total} = g_{data} + \lambda w$$

具体数字例子

设:

$$w = 3, \quad g_{data} = 0.5, \quad \lambda = 0.2$$

那么正则项额外贡献:

$$\lambda w = 0.2 \times 3 = 0.6$$

最终优化器看到的 Gradient:

$$g_{total} = 0.5 + 0.6 = 1.1$$

所以 L2 regularization 并不是一句抽象的“防止过拟合”，它在数学上就是给 **Gradient** 增加一个与当前参数值成正比的项。

3 为什么普通 SGD 里 L2 和 Weight Decay 看起来完全一样?

最基础的 SGD 更新:

$$w_{t+1} = w_t - \eta g_t$$

若加入 L2 penalty:

$$w_{t+1} = w_t - \eta(g_{data} + \lambda w_t)$$

展开:

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta g_{data}$$

看第一项:

$$(1 - \eta\lambda)w_t$$

它的含义就是: 每一步先让参数按比例缩小一点。

例如:

$$\eta = 0.1, \quad \lambda = 0.2$$

则:

$$1 - \eta\lambda = 0.98$$

也就是每一步参数本身先乘 0.98。

为什么叫 Weight Decay?

如果暂时忽略 data gradient:

$$w_{t+1} = (1 - \eta\lambda)w_t$$

那么参数会随 Step 反复乘一个小于 1 的因子, 所以绝对值逐渐“衰减”(decay)。

实验中使用:

$$w = [3, -2], \quad g = [0.5, -1.5], \quad \eta = 0.1, \quad \lambda = 0.2$$

得到:

$$\text{SGD} + \text{L2} = [2.89, -1.81]$$

$$\text{显式 decay 形式} = [2.89, -1.81]$$

最大误差为 0。

非常重要的限定

“L2 regularization 等价于 weight decay”这句话必须带条件。对最基础的 SGD, 这个代数等价关系成立; 但如果优化器会对 Gradient 做 Momentum、自适应缩放、预条件等变换, 事情就不再这么简单。AdamW 就是为这个问题而来。

4 问题出在哪里: Adam 不会原样使用 Gradient

Adam 不是:

$$\theta \leftarrow \theta - \eta g$$

而是先把 Gradient 送进历史状态：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

再形成：

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

如果我们把 L2 项直接加进 Gradient：

$$g_t^{coupled} = g_{data,t} + \lambda \theta_{t-1}$$

那么这个 $\lambda \theta$ 不只是简单缩小参数，而会：

1. 进入 m_t ；
2. 进入 v_t ；
3. 被历史平滑；
4. 被 $\sqrt{v_t}$ 做坐标级缩放。

因此，regularization term 和 Adam 的自适应机制耦合在了一起。

5 最直观的实验：Data Gradient 等于 0

为了把机制看得最清楚，我们故意设：

$$g_{data} = 0$$

并取三个参数：

$$\theta = [10, 1, 0.1]$$

学习率：

$$\eta = 0.01$$

weight decay：

$$\lambda = 0.1$$

Adam + coupled L2

Adam 看到：

$$g = \lambda \theta = [1, 0.1, 0.01]$$

第 1 Step bias correction 后，Adam 对每个坐标大致做：

$$\frac{g}{|g|} \approx 1$$

因此三个正参数第一步都减少约 0.01:

$$[10, 1, 0.1] \rightarrow [9.99, 0.99, 0.090000001]$$

注意：三个参数的绝对变化几乎一样。

AdamW + decoupled weight decay

AdamW 的 decay 直接作用在参数上:

$$\theta \leftarrow (1 - \eta\lambda)\theta$$

这里:

$$1 - \eta\lambda = 0.999$$

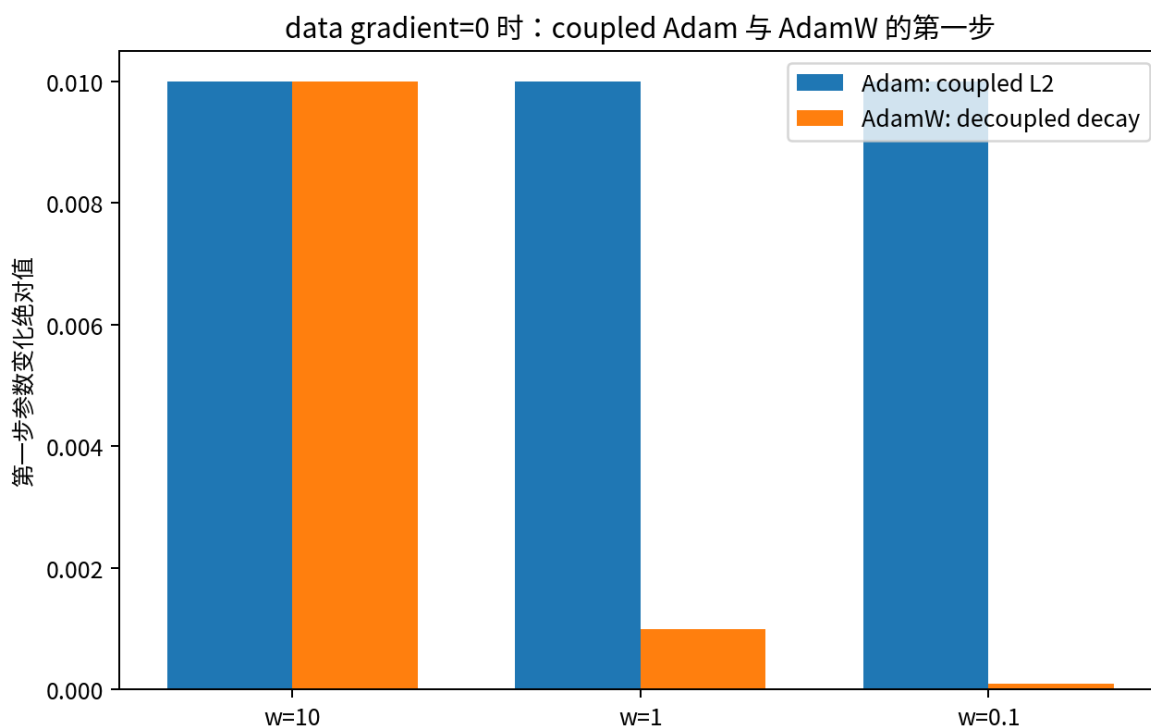
于是:

$$[10, 1, 0.1] \rightarrow [9.99, 0.999, 0.0999]$$

也就是说，变化量分别为:

$$[-0.01, -0.001, -0.0001]$$

严格与参数大小成比例。



这张图是本讲核心

Coupled Adam 中，L2 项先变成 Gradient，再被 Adam 的 m/v 机制处理；AdamW 中，weight decay 直接对 Parameter 做比例收缩，不进入 Adam 的一阶矩和二阶矩估计。这就是 **decoupled weight decay** 的核心。

6 看 Optimizer State：差别更加直接

同样在 $g_{data} = 0$ 的实验里：

Adam 的 state 变成：

$$m = [0.1, 0.01, 0.001]$$

$$v = [0.001, 0.00001, 0.0000001]$$

即使数据没有产生任何 Gradient，weight decay 仍然进入了 Adam 的历史状态。

而 AdamW：

$$m = [0, 0, 0]$$

$$v = [0, 0, 0]$$

因为 data gradient 为 0，Adam 的 moment state 就保持 0；参数收缩由独立的 decay 路径完成。

一句话区分

Adam + L2：“把 regularization 当成 Gradient 的一部分。”

AdamW：“先/同时按比例衰减 Parameter，再用纯 data gradient 做 Adam 自适应更新。”

7 AdamW 的更新公式逐项拆解

概念上可写成：

$$\theta_t = (1 - \eta_t \lambda) \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中 m_t, v_t 只根据 data gradient $g_t = \nabla L_{data}$ 更新。

逐项解释：

- $(1 - \eta_t \lambda) \theta_{t-1}$ ：独立的 weight decay；
- $\hat{m}_t$ ：近期 Gradient 方向历史；
- $\hat{v}_t$ ：近期 Gradient 平方尺度历史；
- η_t ：当前 Optimizer Step 的 Learning Rate；
- λ ：weight decay coefficient；
- ϵ ：Adam 数值稳定项。

注意 decay 强度本身也包含 η_t ：

$$\text{每步 shrink factor} = 1 - \eta_t \lambda$$

所以 Learning Rate schedule 会影响累计 decay。

8 多步纯衰减：AdamW 是可解释的比例收缩

若始终：

$$g_{data} = 0$$

且 LR 恒定，则 AdamW：

$$\theta_T = (1 - \eta\lambda)^T \theta_0$$

实验中：

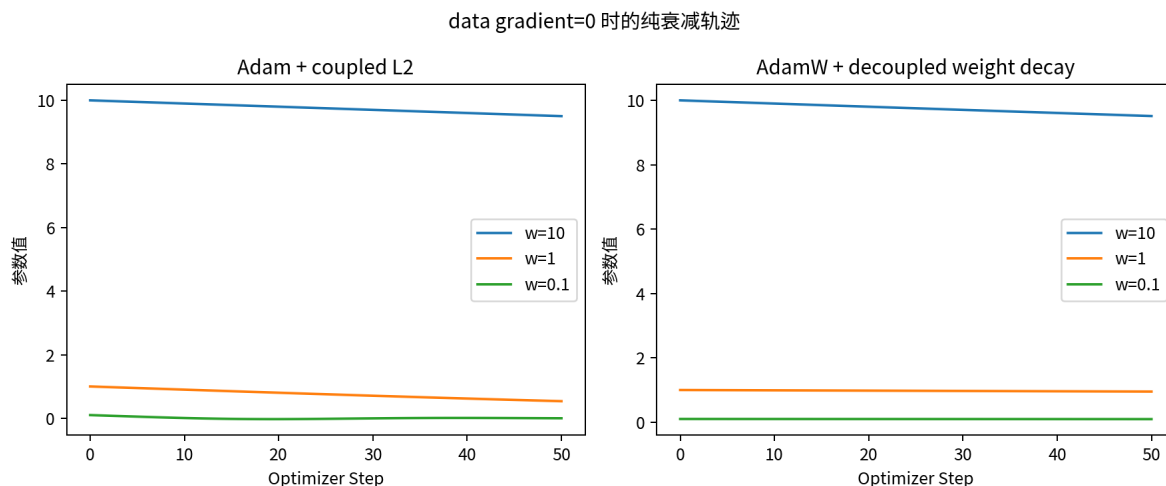
$$\eta = 0.01, \quad \lambda = 0.1, \quad T = 50$$

从 10 开始：

$$10 \times 0.999^{50} \approx 9.512056$$

程序得到完全相同的结果。

而 coupled Adam 的轨迹不是简单的固定比例衰减，甚至初始很小的正参数在若干 Step 后可能跨过 0。这不是 bug，而是因为 L2 Gradient 被 Adam 的自适应历史变换了。



9 L2 Regularization 和 Weight Decay 到底是不是一回事？

现在可以给出严格一点的答案：

场景	做法	关系
基础 SGD	在 loss 加 $\frac{\lambda}{2} \ \theta\ ^2$ vs 参数乘 $(1 - \eta\lambda)$	代数等价
Adam coupled	把 $\lambda\theta$ 加进 Gradient	会进入 m, v ，不再等价于简单比例 decay
AdamW	decay 与 Adam Gradient 更新解耦	参数按比例 shrink，moment state 基于 data gradient

命名上的现实复杂性

不同论文、框架版本、代码库可能把 `weight_decay` 这个参数名用于不同语义。不能只看变量名，要确认 optimizer 的算法定义。在本讲实验使用的 PyTorch 2.10 中：`Adam(..., decoupled_weight_decay=False)` 是 coupled 语义；`AdamW` 是 decoupled 语义；当前版本的 `Adam(..., decoupled_weight_decay=True)` 与 `AdamW` 对齐。为了跨版本代码更清晰，想要 `AdamW` 语义时直接写 `torch.optim.AdamW` 最容易读懂。

10 PyTorch 代码：到底该怎么看？

最典型的 `AdamW`：

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.1,
)
```

看到这段代码，至少要问：

1. LR 是多少？是否有 warmup / cosine decay？
2. weight decay 是多少？
3. 哪些参数真正进入了这个 parameter group？
4. bias、LayerNorm / RMSNorm 参数是否排除了 decay？
5. Gradient Accumulation 后每多少 micro-batch 才调用一次 `optimizer.step()`？

11 Parameter Groups：是不是所有参数都要 Weight Decay？

很多 Transformer / LLM recipe 会把参数分组。例如：

- Linear / projection 的矩阵 Weight：常常施加 weight decay；
- Bias：常常不 decay；
- LayerNorm / RMSNorm 的缩放参数：常常不 decay。

一个概念代码：

```
decay_params = []
no_decay_params = []

for name, p in model.named_parameters():
    if not p.requires_grad:
        continue
```

```
if p.ndim >= 2:
    decay_params.append(p)
else:
    no_decay_params.append(p)

optimizer = torch.optim.AdamW([
    {"params": decay_params, "weight_decay": 0.1},
    {"params": no_decay_params, "weight_decay": 0.0},
], lr=3e-4)
```

常见误区

“二维以上参数 decay，一维参数不 decay”是一种常见工程 recipe，不是数学定律。不同模型、论文和代码库会有不同分组规则。做科研复现时必须以原始实验配置为准，不能凭习惯擅自修改。

12 Learning Rate Schedule 会改变累计 Weight Decay

AdamW 每一步的 decay factor:

$$1 - \eta_t \lambda$$

所以即使 weight decay coefficient λ 完全相同，只要 LR schedule 不同，累计 shrink 就会不同。

实验设置:

$$\lambda = 0.1, \quad T = 100$$

恒定:

$$\eta_t = 0.01$$

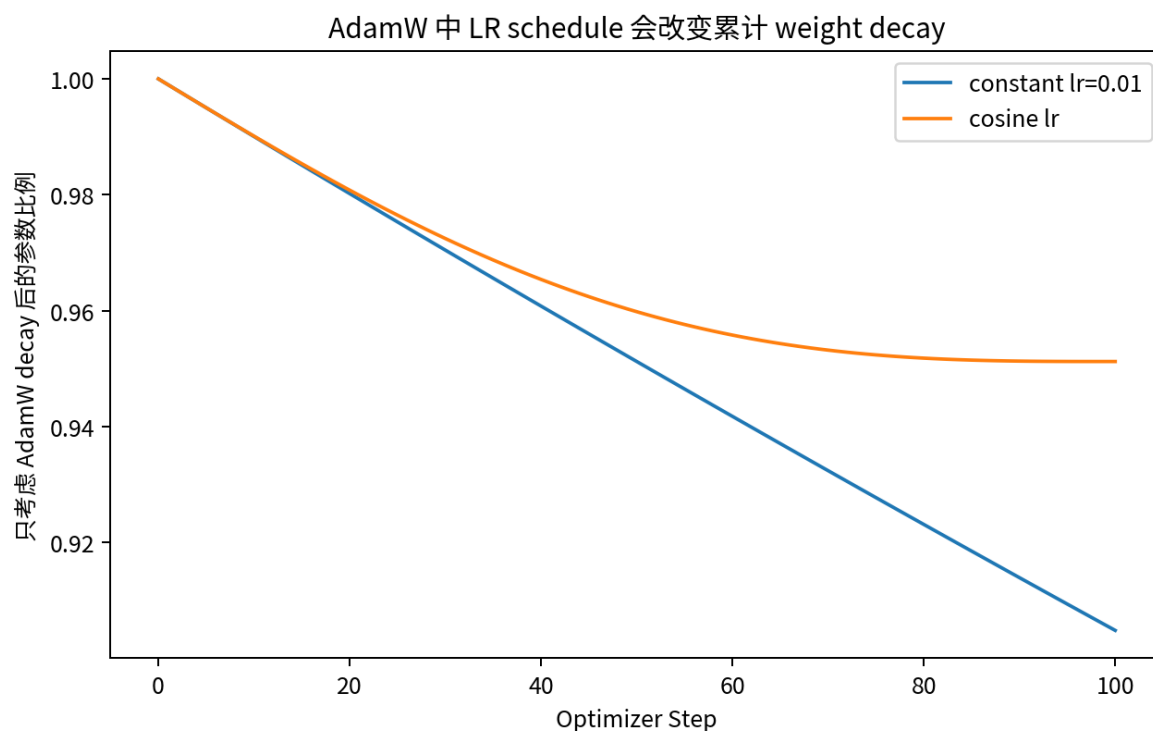
最终纯 decay 比例约:

$$0.90479$$

如果 LR 从 0.01 cosine decay 到接近 0:

$$\theta_T / \theta_0 \approx 0.95121$$

因为后期 LR 变小，weight decay 的单步作用也同步变弱。



科研含义

因此论文里只写“weight decay=0.1”并不能完整描述正则化轨迹。LR、LR schedule、总 optimizer steps、哪些参数被 decay 都会共同决定实际效果。

13 Weight Decay 不是越大越好

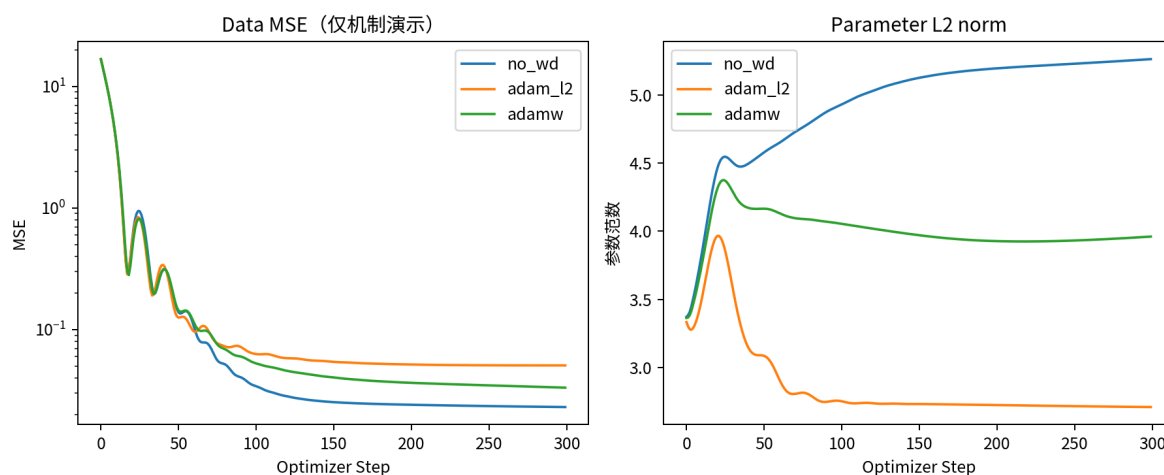
Weight decay 只是一个超参数。如果 λ 太大：

- 参数被持续强行压小；
- 模型可能难以拟合训练数据；
- 出现 underfitting；
- 与 LR / schedule 配合不当时可能改变优化轨迹。

如果太小：其作用可能非常弱。

本讲的小回归机制实验中，同一个初始化：

Optimizer	最终 data MSE	最终参数 L2 norm
Adam, no WD	0.02308	5.2633
Adam, coupled WD	0.05080	2.7085
AdamW	0.03333	3.9603



不要从这个玩具实验推出“谁泛化更好”

这个实验只是展示：相同数值的 weight decay，在 coupled Adam 和 AdamW 中会导致不同的优化与参数范数轨迹。它不能证明 AdamW 在所有任务上都一定有更低 Test Loss，也不能证明参数越小泛化就一定越好。Generalization 必须在独立验证 / 测试数据上实验判断。

14 与 LLM / Transformer 的关系

你以前在 LLM 课程中已经见过 AdamW。现在可以把它放回完整训练链：

Token batch $\rightarrow$ Transformer $\rightarrow$ Cross Entropy $\rightarrow$ Backprop $\rightarrow$ Gradient $\rightarrow$ Adam moments $\rightarrow$ AdamW update + decay

LLM 训练中常见 AdamW，并不意味着只要看到 Transformer 就必须固定某个 weight decay 数值。实际 recipe 还会与：

- 模型规模；
- 数据规模与训练 token 数；
- Learning Rate；
- Warmup / Cosine schedule；
- Batch / Gradient Accumulation；
- Parameter Group；
- 训练阶段（pretraining / finetuning）

共同决定。

15 与 Agent Systems 科研的关系

Agent Systems 研究中，如果你只是通过 API 调用闭源 LLM，AdamW 可能不在你的实验控制范围内。但如果进行：

- LoRA / QLoRA;
- 小模型 SFT;
- Reward Model 训练;
- Policy / value model 微调;
- Agent memory encoder 或 reranker 训练;
- 本地小模型行为机制实验,

那么 optimizer / LR / weight decay 都会成为实验条件。

科研记录建议

至少记录: optimizer 类型、lr、betas、eps、weight decay、parameter groups、scheduler、warmup、batch / accumulation、optimizer steps、random seed。否则别人很难判断实验差异到底来自模型方法, 还是来自训练 recipe。

16 Weight Decay 在 Training Loop 的哪一步发生?

把 AdamW 放回你已经学过的 PyTorch Training Loop:

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

这里要精确区分:

- `forward()`: 只根据当前 Parameter 计算预测;
- `loss.backward()`: 计算 data loss 对 Parameter 的 Gradient;
- `optimizer.step()`: AdamW 才真正执行 Adam 更新与 decoupled weight decay;
- `model.eval()`: 不会自动关闭 weight decay, 因为只要你不调用 `optimizer.step()`, 参数就不会被 optimizer 修改。

因此 weight decay 不是一个“推理时额外插进去的层”, 也不会在每次 `model(x)` 时自动把参数变小。它属于**训练优化规则**。

为什么这个区别很重要

假设你只做:

```
with torch.no_grad():
    y = model(x)
```

即使 optimizer 是 AdamW(weight_decay=0.1), Parameter 也不会因为这次推理而衰减。只有真正执行 optimizer step, 才发生参数更新。

17 AdamW 的 State、Checkpoint 与断点续训

AdamW 与 Adam 一样，需要为 Parameter 保存：

$$m_t, \quad v_t, \quad step$$

Decoupled weight decay 本身通常不需要再增加一套和 Parameter 同 Shape 的额外状态；但这并不意味着只保存 model weights 就足够继续训练。

完整断点通常至少包含：

```
checkpoint = {  
    "model": model.state_dict(),  
    "optimizer": optimizer.state_dict(),  
    "scheduler": scheduler.state_dict(),  
    "global_step": global_step,  
}
```

若只恢复 model weights，却重新创建一个全新的 AdamW：

- m_t, v_t 会重新从 0 开始；
- bias correction 的 step 计数重新开始；
- LR scheduler 若未恢复也可能回到错误位置；
- 后续训练轨迹不再等价于原训练的连续延伸。

LLM 训练中的现实含义

大模型训练经常运行很久并跨多个 checkpoint。Optimizer state 甚至可能占用与模型参数同量级或更高的内存。因此“能加载模型继续推理”和“能严格恢复训练”是两件不同的事。

18 怎样设计 Adam vs AdamW 的公平 Ablation?

以后如果论文声称“AdamW 优于 Adam”，或者你自己要比较 optimizer，至少应该控制以下变量：

1. 相同模型初始化或相同 random seed；
2. 相同数据顺序 / DataLoader seed；
3. 相同 batch、gradient accumulation、总 optimizer steps；
4. 相同 nominal LR 与 scheduler；
5. 相同 $\beta_1, \beta_2, \epsilon$ ；
6. 清楚说明 coupled Adam 的 weight decay 与 AdamW 的 decoupled weight decay 是否使用相同数值；
7. parameter groups 完全一致；
8. 同时报告 Training Loss、Validation/Test 指标和 Parameter Norm，而不是只挑其中一个；

9. 多随机种子时报告均值与波动，避免把一次随机结果当成普遍规律。

更严格地说，相同数值的 λ 在 coupled Adam 和 AdamW 中**不是相同的数学干预**，所以“都设成 0.1”只能表示超参数数字一样，不能表示正则化强度完全可比。真正公平的比较有时需要分别调参，或者明确研究问题究竟是“默认 recipe 比较”还是“机制比较”。

Ablation 在测什么？

如果只把 Adam 改成 AdamW，其余完全不变，这个 ablation 测的是“optimizer + regularization coupling 机制改变后的整体效果”。如果还同时修改 LR、scheduler、parameter groups，就不能再把结果简单归因于 AdamW 本身。

19 常见误区集中纠正

误区 1：L2 和 AdamW 就是同一件事

不对。基础 SGD 中有等价关系；Adam 的自适应预条件会破坏这种简单等价，AdamW 则把 decay 解耦出来。

误区 2：weight decay=0.1 就表示每步参数乘 0.9

不对。AdamW 中每步大致乘：

$$1 - \eta_t \lambda$$

还要乘 Learning Rate。若 $\eta = 3 \times 10^{-4}$ 、 $\lambda = 0.1$ ，单步 factor 约为 0.99997，不是 0.9。

误区 3：Weight Decay 会把所有参数迅速变成 0

不一定。真实训练同时有 data gradient 在推参数，并且 LR 通常会变化；最终是数据驱动更新与 decay 的长期平衡。

误区 4：Weight norm 越小，模型就一定越好

不对。Parameter norm 只是一个量。泛化、校准、鲁棒性和下游行为必须独立评估。

20 本讲最低闭环

如果只保留最关键的六句话：

1. L2 penalty: $J = L_{data} + \frac{\lambda}{2} \|\theta\|^2$;
2. 它给 Gradient 增加 $\lambda\theta$;
3. 基础 SGD 中，这等价于每步把参数乘 $(1 - \eta\lambda)$;
4. Adam 会加工 Gradient，因此 coupled L2 不再等价于简单比例 decay;
5. AdamW 把 Parameter decay 与 Adam 的 moment estimation 解耦;
6. 实际 decay 强度还取决于 LR、schedule、steps 和 parameter groups。

下一讲预告

下一讲进入第 40 讲：Initialization、Vanishing Gradient 与 Exploding Gradient。我们会回答：神经网络最开始的 Weight 为什么不能全设成 0？为什么 Xavier / He 初始化会考虑输入维度？为什么深层网络 Gradient 会越来越小或越来越大？以及 Gradient Clipping 到底是在“治病”还是“止血”。